FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Pragamac, Renzel John S.
Section:	AN43
Professor:	Crisola G. Tan
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

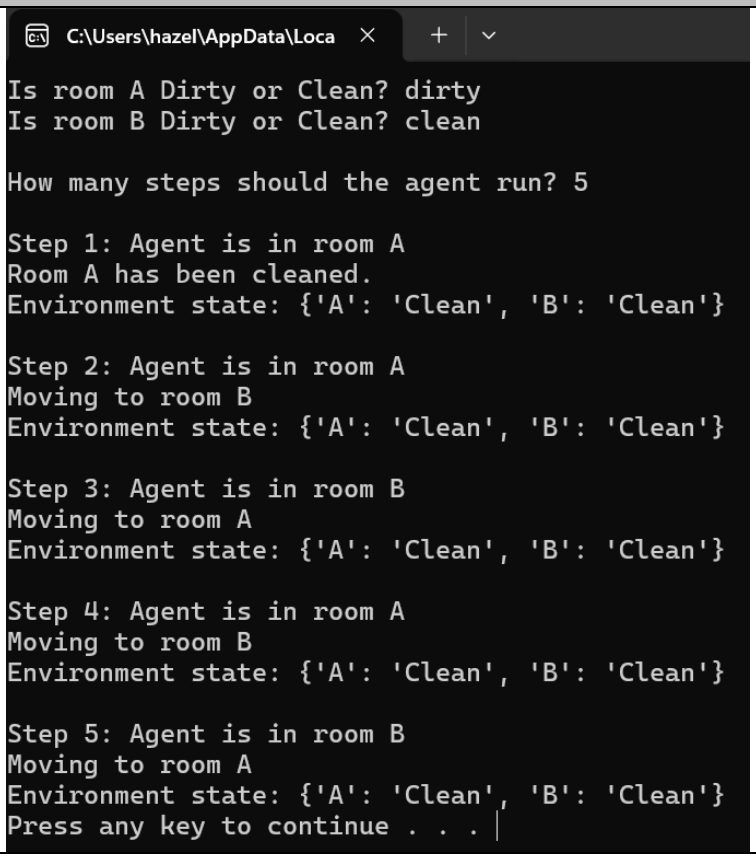
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

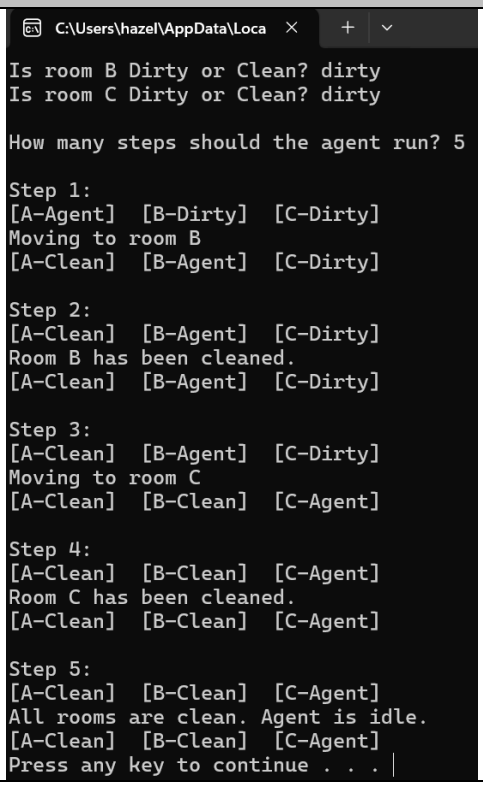
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks

CODE

```
import random

class VacuumAgent:
    def __init__(self, start_location='A'):
        self.location = start_location

    def display_grid(self, environment):
        grid = []
        for room in ['A', 'B', 'C']:
            if room == self.location:
                grid.append(f"[{room}-Agent]")
            else:
                grid.append(f"[{room}-{environment[room]}]")
        print(" ".join(grid))

    def perceive_and_act(self, environment):
        if environment[self.location].lower() == 'dirty':
            environment[self.location] = 'Clean'
            print(f"Room {self.location} has been cleaned.")
            self.display_grid(environment)
        else:
            if all(state.lower() == 'clean' for state in environment.values()):
                print("All rooms are clean. Agent is idle.")
                self.display_grid(environment)
            else:
                other_rooms = [r for r in ['A', 'B', 'C'] if r != self.location]
                self.location = random.choice(other_rooms)
                print(f"Moving to room {self.location}")
                self.display_grid(environment)

def main():
    # Task 1: Set Up Environment for Rooms A, B, and C
    environment = {}
    environment['A'] = input("Is room A Dirty or Clean? ").strip().capitalize()
    environment['B'] = input("Is room B Dirty or Clean? ").strip().capitalize()
    environment['C'] = input("Is room C Dirty or Clean? ").strip().capitalize()

    # Task 2: Implement the Rule-Based Agent
    agent = VacuumAgent(start_location='A')

    # Task 3: Run the Simulation
```

```
    steps = int(input("How many steps should the agent run? "))

    for step in range(1, steps + 1):
        print(f"\nStep {step}:")
        agent.display_grid(environment)
        agent.perceive_and_act(environment)

    print("\nSimulation completed yippie")

if __name__ == "__main__":
    main()
```

```
Is room A Dirty or Clean? Clean
Is room B Dirty or Clean? Dirty
Is room C Dirty or Clean? Dirty
How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]

Simulation completed yippie
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent cleans the current room if it is dirty, and moves to the other room if the current room is clean.
2. The agent decides by checking the state of its current room in the environment, triggering a clean action if dirty or a move action if clean.
3. The agent will continuously move back and forth between the two clean rooms for the entire duration of the simulation.
4. You can modify the agent by replacing its simple two-room toggle with dynamic room lists and implementing strategies like sequential movement, random selection, or memory-based pathfinding to navigate multiple rooms.